

# PyGNSS-RT

## Technical Design Document

Python-Based Real-Time PPP-AR GNSS Positioning System

Technical Specification and Architecture

**Repository:** [https://github.com/georgisalene/pygnss\\_rt](https://github.com/georgisalene/pygnss_rt)

**Version:** v1.4.0-GRE

**Configuration:** GPS + GLONASS + Galileo (GRE)

**Python Files:** 75 modules across 10 packages

**Database:** DuckDB (embedded analytical database)

**Addisu Hunegnaw**

GNSS Research Group

Institute for Earth Sciences and Space Geodesy

`pygnss_rt` — Python GNSS Real-Time Processing

January 2026

## Abstract

This document provides a comprehensive technical specification for the `pygnss_rt` Python package, a production-grade framework for Real-Time Precise Point Positioning with Ambiguity Resolution (PPP-AR). The system integrates with the Bernese GNSS Software v5.4 for core geodetic computations while providing a sophisticated Python orchestration layer for data management, product acquisition, and operational automation.

### Key Features:

- **Multi-GNSS:** GPS (G) + GLONASS (R) + Galileo (E) triple-constellation
- **Troposphere:** VMF3 Vienna Mapping Functions for ZTD modeling
- **Biases:** CODE OSB (Observable-Specific Signal Biases) for PPP-AR
- **AR:** LAMBDA-based integer ambiguity resolution
- **Database:** DuckDB embedded analytical database
- **Downloads:** Multi-source FTP/HTTPS from CDDIS, CODE, IGN
- **Networks:** Five international networks (IGS, EUREF, GB, RGP, Supersites)

**GitHub Repository:** [https://github.com/georgisalene/pygnss\\_rt](https://github.com/georgisalene/pygnss_rt)

The package replaces legacy Perl scripts with modern Python, providing improved maintainability, type safety, and extensibility while maintaining full compatibility with the Bernese GNSS Software v5.4 processing engine.

# Contents

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                     | <b>5</b>  |
| 1.1      | Document Purpose and Scope . . . . .    | 5         |
| 1.1.1    | What This Document Provides . . . . .   | 5         |
| 1.2      | Features Converted from Perl . . . . .  | 5         |
| 1.3      | Networks Supported . . . . .            | 6         |
| <b>2</b> | <b>Project Structure</b>                | <b>7</b>  |
| 2.1      | Complete Package Architecture . . . . . | 7         |
| 2.2      | Directory Tree Structure . . . . .      | 7         |
| 2.3      | Module Statistics . . . . .             | 9         |
| <b>3</b> | <b>Installation</b>                     | <b>10</b> |
| 3.1      | System Requirements . . . . .           | 10        |
| 3.2      | Installation from GitHub . . . . .      | 10        |
| 3.3      | Python Dependencies . . . . .           | 10        |
| 3.4      | Initialize Database . . . . .           | 11        |
| <b>4</b> | <b>Database System (DuckDB)</b>         | <b>12</b> |
| 4.1      | Why DuckDB? . . . . .                   | 12        |
| 4.2      | Database Schema . . . . .               | 12        |
| 4.2.1    | Products Table . . . . .                | 12        |
| 4.2.2    | Hourly Data Table . . . . .             | 13        |
| 4.2.3    | ZTD Results Table . . . . .             | 13        |
| 4.2.4    | Hourly MET Table . . . . .              | 13        |
| 4.3      | Database Operations . . . . .           | 14        |
| <b>5</b> | <b>MET Data Maintenance</b>             | <b>15</b> |
| 5.1      | Purpose . . . . .                       | 15        |
| 5.2      | MET Status Flow . . . . .               | 15        |
| 5.3      | MetManager Class . . . . .              | 15        |
| 5.4      | Cron Integration . . . . .              | 16        |
| <b>6</b> | <b>Daily PPP Processing</b>             | <b>17</b> |
| 6.1      | Processing Workflow . . . . .           | 17        |
| 6.2      | Running Daily PPP . . . . .             | 17        |
| 6.3      | Command Options . . . . .               | 18        |
| 6.4      | Result Structure . . . . .              | 18        |
| <b>7</b> | <b>FTP/Download System</b>              | <b>19</b> |
| 7.1      | Multi-Source Architecture . . . . .     | 19        |
| 7.2      | FTP Configuration (YAML) . . . . .      | 19        |
| 7.3      | Supported Products . . . . .            | 20        |
| 7.4      | Station Data Sources . . . . .          | 20        |

|           |                                           |           |
|-----------|-------------------------------------------|-----------|
| <b>8</b>  | <b>Date/Time Handling</b>                 | <b>21</b> |
| 8.1       | GNSSDate Class . . . . .                  | 21        |
| 8.2       | Time System Constants . . . . .           | 21        |
| 8.3       | GNSSDate Properties and Methods . . . . . | 22        |
| 8.4       | Hour Alpha Mapping . . . . .              | 22        |
| <b>9</b>  | <b>Station Management</b>                 | <b>23</b> |
| 9.1       | Station Class . . . . .                   | 23        |
| 9.2       | Station File Formats . . . . .            | 23        |
| 9.3       | Station Commands . . . . .                | 23        |
| <b>10</b> | <b>Troposphere Processing</b>             | <b>25</b> |
| 10.1      | ZTD to IWV Conversion . . . . .           | 25        |
| 10.1.1    | Mathematical Formulation . . . . .        | 25        |
| 10.2      | Physical Constants . . . . .              | 25        |
| 10.3      | ZTD2IWV Command . . . . .                 | 25        |
| <b>11</b> | <b>Bernese Configuration</b>              | <b>27</b> |
| 11.1      | GRE Mode Configuration . . . . .          | 27        |
| 11.2      | GRE Configuration Rationale . . . . .     | 28        |
| <b>12</b> | <b>Mathematical Foundations</b>           | <b>29</b> |
| 12.1      | PPP-AR Observation Model . . . . .        | 29        |
| 12.2      | State Vector . . . . .                    | 29        |
| 12.3      | LAMBDA Algorithm . . . . .                | 29        |
| 12.4      | VMF3 Mapping Functions . . . . .          | 29        |
| <b>A</b>  | <b>CLI Command Reference</b>              | <b>30</b> |
| <b>B</b>  | <b>References</b>                         | <b>31</b> |

# List of Figures

|     |                                                   |    |
|-----|---------------------------------------------------|----|
| 1.1 | Supported Station Networks . . . . .              | 6  |
| 2.1 | PyGNSS-RT Complete Package Architecture . . . . . | 7  |
| 4.1 | DuckDB Schema Overview . . . . .                  | 12 |
| 5.1 | MET File Status Transitions . . . . .             | 15 |
| 6.1 | Daily PPP Processing Workflow . . . . .           | 17 |
| 7.1 | Multi-Source Download Architecture . . . . .      | 19 |
| 8.1 | GNSSDate Time Representations . . . . .           | 21 |

# List of Tables

|      |                                             |    |
|------|---------------------------------------------|----|
| 1.1  | Perl to Python Conversion Summary . . . . . | 5  |
| 1.2  | Network Configuration Details . . . . .     | 6  |
| 2.1  | Package Module Count . . . . .              | 9  |
| 3.1  | System Requirements . . . . .               | 10 |
| 3.2  | Python Package Dependencies . . . . .       | 11 |
| 4.1  | DatabaseManager Methods . . . . .           | 14 |
| 5.1  | MetManager Methods . . . . .                | 15 |
| 6.1  | daily-ppp Command Options . . . . .         | 18 |
| 6.2  | DailyPPPPResult Fields . . . . .            | 18 |
| 7.1  | GNSS Product Types . . . . .                | 20 |
| 7.2  | Station Data Servers . . . . .              | 20 |
| 8.1  | GNSS Time Constants . . . . .               | 21 |
| 8.2  | GNSSDate Class API . . . . .                | 22 |
| 8.3  | Hour to Alpha Character Mapping . . . . .   | 22 |
| 9.1  | Station Class Attributes . . . . .          | 23 |
| 9.2  | Supported Station File Formats . . . . .    | 23 |
| 10.1 | Physical Constants (Bevis, 1994) . . . . .  | 25 |
| 11.1 | GRE Processing Strategy . . . . .           | 28 |
| A.1  | Complete CLI Commands . . . . .             | 30 |

# Chapter 1

## Introduction

### 1.1 Document Purpose and Scope

This technical design document serves as the authoritative reference for the `pygnss_rt` real-time GNSS processing system. The complete source code is available at:

#### GitHub Repository

[https://github.com/georgisalene/pygnss\\_rt](https://github.com/georgisalene/pygnss_rt)

#### 1.1.1 What This Document Provides

- **Complete Project Structure** – 75 Python modules across 10 packages
- **Installation Guide** – pip-based installation and dependencies
- **Database Architecture** – DuckDB schema and rationale
- **Processing Workflows** – Daily PPP, troposphere, MET data
- **FTP/Download System** – Multi-source product acquisition
- **Configuration Reference** – BSW YAML configuration format

### 1.2 Features Converted from Perl

The `pygnss_rt` package is a complete port of the legacy Perl i-GNSS system:

Table 1.1: Perl to Python Conversion Summary  
2lightblue!30white

| Perl Module/Script          | Python Module                     | Purpose               |
|-----------------------------|-----------------------------------|-----------------------|
| iGNSS_D_PPP_AR_*_NRT.pl (5) | processing/daily_ppp.py           | Daily PPP processing  |
| FTP.pm                      | data_access/product_downloader.py | Product downloads     |
| startBPE.pm, RUNBPE.pm      | bsw/bpe_runner.py                 | BPE execution         |
| DB.pm                       | database/connection.py            | Database operations   |
| call_MET_maintain.pl        | database/met.py                   | MET data tracking     |
| DATES.pm, GMTTIME.pm        | utils/dates.py                    | Date utilities        |
| ZTD2IWV.pm                  | atmosphere/ztd2iwv.py             | ZTD to IWV conversion |
| ftpConfig.xml               | config/ftp_servers.yaml           | FTP configuration     |
| XML station configs         | YAML station files                | Station management    |

1.3 Networks Supported

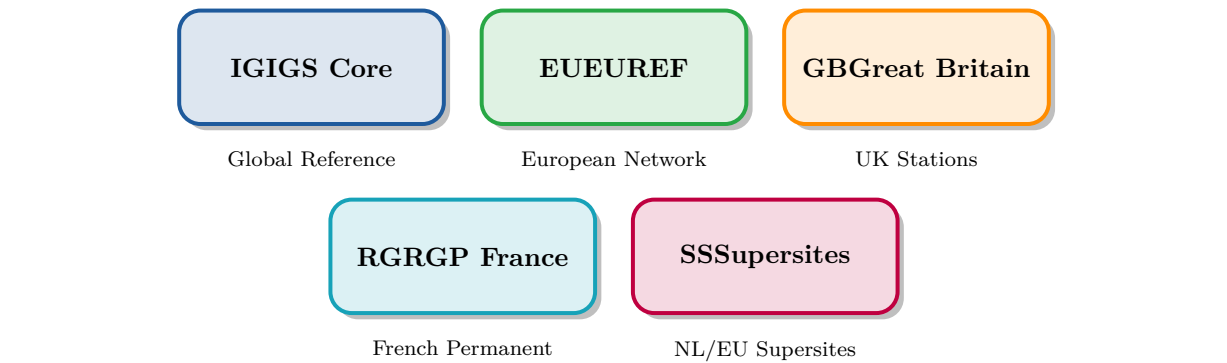


Figure 1.1: Supported Station Networks

| Table 1.2: Network Configuration Details |               |          |             |                             |
|------------------------------------------|---------------|----------|-------------|-----------------------------|
| ID                                       | Name          | Stations | Data Source | Description                 |
| IG                                       | IGS           | ~500     | CDDIS, BKG  | Global IGS core network     |
| EU                                       | EUREF         | ~350     | BKG EUREF   | European reference frame    |
| 2lightgreen!30whiteGB                    | Great Britain | ~120     | OSGB, CDDIS | UK OS Active + scientific   |
| RG                                       | RGP France    | ~450     | RGP IGN     | French permanent network    |
| SS                                       | Supersites    | ~20      | Multiple    | High-quality research sites |

## Chapter 2

# Project Structure

### 2.1 Complete Package Architecture

The `pygnss_rt` package contains **75 Python modules** organized into **10 sub-packages**:

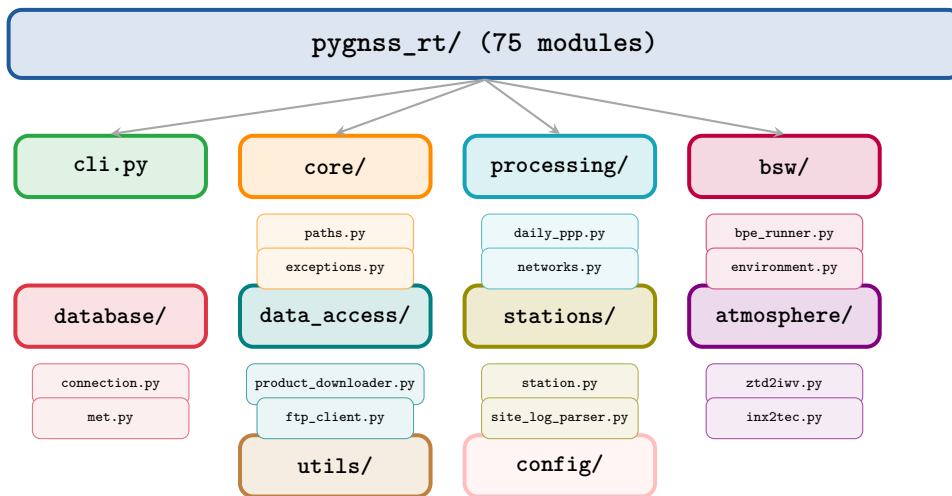


Figure 2.1: PyGNSS-RT Complete Package Architecture

### 2.2 Directory Tree Structure

Listing 2.1: Complete Directory Structure

```
1 pygnss_rt/                                # Root package (75 Python files)
2 |-- __init__.py                            # Package initialization
3 |-- cli.py                                # Command-line interface (Click-based)
4 |-- cron_hourly.py                         # Hourly cron job handler
5 |
6 |-- core/                                  # Core infrastructure
7 |   |-- paths.py                           # Centralized path configuration
8 |   |-- config.py                           # System configuration
9 |   |-- exceptions.py                       # Custom exception classes
10 |
11 |-- processing/                            # GNSS processing modules
12 |   |-- daily_ppp.py                       # Daily PPP-AR processor
13 |   |-- networks.py                        # Network profile definitions
14 |   |-- orchestrator.py                    # Processing orchestration
15 |   |-- station_merger.py                  # Station file merging
16 |   |-- daily_crd.py                       # Daily coordinate processing
17 |
18 |-- bsw/                                   # Bernese Software integration
19 |   |-- bpe_runner.py                      # BPE execution wrapper
20 |   |-- environment.py                     # BSW environment setup
21 |
```



```

22 |-- database/                # DuckDB database layer
23 |   |-- connection.py       # DatabaseManager class
24 |   |-- met.py              # MET data maintenance
25 |   |-- products.py         # Product tracking
26 |   |-- hourly_data.py      # Hourly data tracking
27 |   |-- models.py           # Data models
28 |
29 |-- data_access/           # Data acquisition
30 |   |-- product_downloader.py # GNSS product downloads
31 |   |-- station_downloader.py # Station RINEX downloads
32 |   |-- ftp_client.py        # FTP/SFTP client
33 |   |-- http_client.py       # HTTPS client
34 |   |-- ftp_config.py        # FTP configuration loader
35 |   |-- met_downloader.py     # MET file downloads
36 |
37 |-- stations/              # Station management
38 |   |-- station.py           # Station class
39 |   |-- site_log_parser.py   # IGS site log parser
40 |   |-- auto_station.py      # Automatic station discovery
41 |
42 |-- atmosphere/            # Atmospheric processing
43 |   |-- ztd2iwv.py           # ZTD to IWV conversion
44 |   |-- inx2tec.py           # IONEX to TEC conversion
45 |
46 |-- utils/                 # Utility functions
47 |   |-- dates.py             # GNSS date/time handling
48 |   |-- logging.py           # Structured logging (structlog)
49 |
50 |-- config/                # Configuration files
51 |   |-- ftp_servers.yaml     # FTP server definitions
52 |
53 |-- bsw_configs/           # BSW YAML configurations
54 |   |-- iGNSS_D_PPP_AR_IG_IGS54_direct.yaml
55 |   |-- iGNSS_D_PPP_AR_EU_IGS54_direct.yaml
56 |   |-- iGNSS_D_PPP_AR_GB_IGS54_direct.yaml
57 |   |-- iGNSS_D_PPP_AR_RG_IGS54_direct.yaml
58 |   |-- iGNSS_D_PPP_AR_SS_IGS54_direct.yaml

```

2.3 Module Statistics

Table 2.1: Package Module Count

| Package      | Color  | Modules | Purpose           |
|--------------|--------|---------|-------------------|
| pygnss_rt/   | Blue   | 3       | Root CLI and cron |
| core/        | Orange | 4       | Infrastructure    |
| processing/  | Cyan   | 8       | GNSS processing   |
| bsw/         | Purple | 3       | BSW integration   |
| database/    | Red    | 8       | DuckDB layer      |
| data_access/ | Teal   | 10      | Downloads         |
| stations/    | Olive  | 5       | Station mgmt      |
| atmosphere/  | Violet | 3       | Atmospheric       |
| utils/       | Brown  | 4       | Utilities         |
| config/      | Pink   | 2       | Configuration     |
| Total        |        | 75      |                   |

# Chapter 3

## Installation

### 3.1 System Requirements

Table 3.1: System Requirements

|                     | Requirement           | Version      | Notes                          |
|---------------------|-----------------------|--------------|--------------------------------|
| 2lightgreen!30white | Python                | $\geq 3.10$  | Python 3.10, 3.11, or 3.12     |
|                     | Bernese GNSS Software | 5.4          | Required for GPSEST processing |
|                     | Operating System      | Linux        | Tested on Ubuntu 22.04+        |
|                     | Disk Space            | $\geq 50$ GB | For products and results       |
|                     | Memory                | $\geq 8$ GB  | For large network processing   |

### 3.2 Installation from GitHub

Listing 3.1: Installation Commands

```
1 # Clone the repository
2 git clone https://github.com/georgisalene/pygnss_rt.git
3 cd pygnss_rt
4
5 # Create virtual environment (recommended)
6 python3 -m venv .venv
7 source .venv/bin/activate
8
9 # Install in development mode
10 pip install -e .
11
12 # Or install with development dependencies
13 pip install -e ".[dev]"
14
15 # Verify installation
16 pygnss-rt --version
17 pygnss-rt --help
```

### 3.3 Python Dependencies

The package dependencies are defined in `pyproject.toml`:

Table 3.2: Python Package Dependencies

|                      | Package   | Version       | Purpose                       |
|----------------------|-----------|---------------|-------------------------------|
|                      | duckdb    | $\geq 1.0.0$  | Embedded analytical database  |
|                      | pyyaml    | $\geq 6.0$    | YAML configuration parsing    |
|                      | paramiko  | $\geq 3.0.0$  | SSH/SFTP for secure transfers |
|                      | astropy   | $\geq 6.0.0$  | Time system conversions       |
| 2lightorange!30white | numpy     | $\geq 1.24.0$ | Numerical computations        |
|                      | pandas    | $\geq 2.0.0$  | Data manipulation             |
|                      | click     | $\geq 8.0.0$  | Command-line interface        |
|                      | structlog | $\geq 24.0.0$ | Structured logging            |
|                      | lxml      | $\geq 5.0.0$  | XML parsing (legacy support)  |
|                      | requests  | $\geq 2.31.0$ | HTTPS downloads               |
|                      | pydantic  | $\geq 2.0.0$  | Data validation               |

## 3.4 Initialize Database

After installation, initialize the DuckDB database:

Listing 3.2: Database Initialization

```

1 # Initialize database with default schema
2 pygnss-rt init-db
3
4 # Or specify custom database path
5 pygnss-rt init-db --db-path /path/to/pygnss-rt.duckdb
6
7 # Verify database
8 pygnss-rt db-status

```

## Chapter 4

# Database System (DuckDB)

### 4.1 Why DuckDB?

**DuckDB** is an embedded analytical database designed for OLAP workloads. Unlike traditional databases (PostgreSQL, MySQL), DuckDB:

- **Runs in-process** – No server setup required
- **Single file** – Entire database in one `.duckdb` file
- **Column-oriented** – Optimized for analytical queries
- **Fast aggregations** – Ideal for time-series GNSS data
- **Zero dependencies** – Embedded in Python process

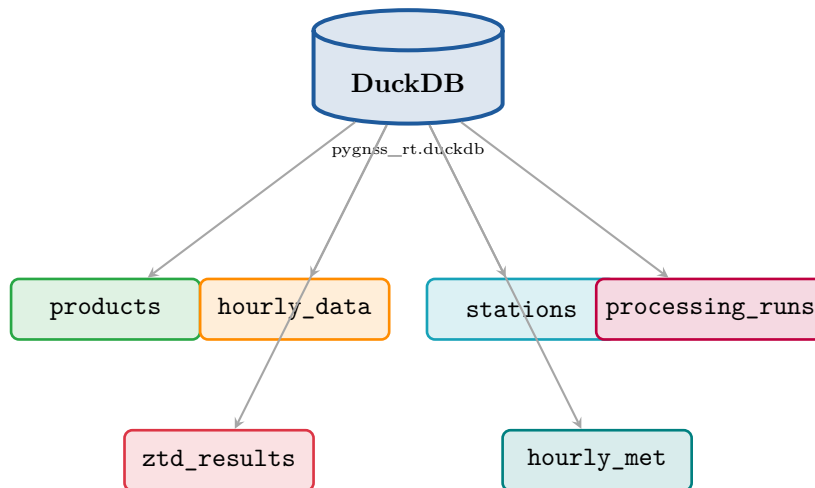


Figure 4.1: DuckDB Schema Overview

### 4.2 Database Schema

#### 4.2.1 Products Table

Tracks downloaded GNSS products (orbits, clocks, biases):

Listing 4.1: Products Table Schema

```
1 CREATE TABLE IF NOT EXISTS products (  
2     id INTEGER PRIMARY KEY,  
3     product_type VARCHAR NOT NULL, -- 'orbit', 'clock', 'erp', 'bia'  
4     provider VARCHAR NOT NULL,    -- 'CDDIS', 'CODE', 'IGN'  
5     tier VARCHAR NOT NULL,        -- 'final', 'rapid', 'ultra'  
6     mjd DOUBLE NOT NULL,         -- Modified Julian Date  
7     gps_week INTEGER,  
8     day_of_week INTEGER,
```

```

9     filename VARCHAR NOT NULL,
10    local_path VARCHAR,
11    file_size BIGINT,
12    checksum VARCHAR,
13    download_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
14    UNIQUE(product_type, provider, tier, mjd)
15 );

```

### 4.2.2 Hourly Data Table

Tracks hourly RINEX observation files:

Listing 4.2: Hourly Data Table Schema

```

1 CREATE TABLE IF NOT EXISTS hourly_data (
2     id INTEGER PRIMARY KEY,
3     station_id VARCHAR NOT NULL,    -- 4-char station code
4     mjd DOUBLE NOT NULL,
5     hour INTEGER NOT NULL,         -- 0-23
6     rinex_file VARCHAR,
7     status VARCHAR DEFAULT 'pending', -- pending/downloaded/processed
8     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
9     UNIQUE(station_id, mjd, hour)
10 );

```

### 4.2.3 ZTD Results Table

Stores PPP-derived troposphere results:

Listing 4.3: ZTD Results Table Schema

```

1 CREATE TABLE IF NOT EXISTS ztd_results (
2     id INTEGER PRIMARY KEY,
3     station_id VARCHAR NOT NULL,
4     mjd DOUBLE NOT NULL,
5     ztd DOUBLE NOT NULL,           -- Zenith Total Delay [mm]
6     ztd_sigma DOUBLE,              -- ZTD sigma [mm]
7     zhd DOUBLE,                    -- Zenith Hydrostatic Delay
8     zwd DOUBLE,                    -- Zenith Wet Delay
9     iwv DOUBLE,                    -- Integrated Water Vapor [kg/m2]
10    iwv_sigma DOUBLE,
11    temperature DOUBLE,             -- Surface temperature [K]
12    pressure DOUBLE,                -- Surface pressure [hPa]
13    processing_run_id INTEGER,
14    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
15    UNIQUE(station_id, mjd)
16 );

```

### 4.2.4 Hourly MET Table

Tracks meteorological data for ZTD to IWV conversion:

Listing 4.4: Hourly MET Table Schema

```

1 CREATE TABLE IF NOT EXISTS hourly_met (
2     year VARCHAR(4) NOT NULL,      -- '2025'
3     doy INTEGER NOT NULL,          -- Day of year 1-366
4     hour VARCHAR(1) NOT NULL,      -- Hour alpha 'a'-'x'

```

```
5   mjd DOUBLE NOT NULL ,
6   status VARCHAR(30) DEFAULT 'Waiting',  -- Waiting/On Time/Too Late
7   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ,
8   updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ,
9   PRIMARY KEY (year, doy, hour)
10 );
```

4.3 Database Operations

Table 4.1: DatabaseManager Methods

| Method                       | Parameters                                    | Description                      |
|------------------------------|-----------------------------------------------|----------------------------------|
| <code>__init__()</code>      | <code>db_path</code> , <code>read_only</code> | Initialize connection            |
| <code>create_schema()</code> | —                                             | Create all tables                |
| <code>execute()</code>       | <code>query</code> , <code>params</code>      | Execute SQL query                |
| <code>fetchone()</code>      | <code>query</code> , <code>params</code>      | Fetch single row                 |
| <code>fetchall()</code>      | <code>query</code> , <code>params</code>      | Fetch all rows                   |
| <code>transaction()</code>   | —                                             | Context manager for transactions |
| <code>close()</code>         | —                                             | Close connection                 |

## Chapter 5

# MET Data Maintenance

### 5.1 Purpose

Meteorological (MET) data provides surface temperature and pressure values needed to convert GNSS-derived Zenith Total Delay (ZTD) to Integrated Water Vapor (IWV). The MET maintenance system:

- Tracks hourly MET file availability
- Manages file download status (Waiting → On Time → Too Late)
- Fills gaps from system interruptions
- Cleans up old entries to prevent table bloat

### 5.2 MET Status Flow

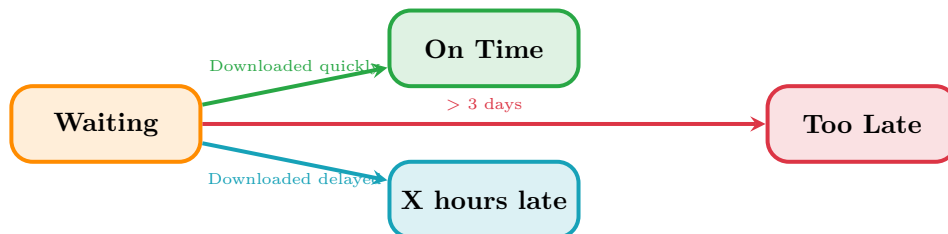


Figure 5.1: MET File Status Transitions

### 5.3 MetManager Class

The `database/met.py` module provides the `MetManager` class:

Table 5.1: MetManager Methods

| Method                                     | Description                              |
|--------------------------------------------|------------------------------------------|
| <code>create_table()</code>                | Create the <code>hourly_met</code> table |
| <code>maintain(date)</code>                | Add new hourly entry for current hour    |
| <code>fill_gap(late_day, late_hour)</code> | Fill gaps from interruptions             |
| <code>set_too_late_files()</code>          | Mark old waiting files as Too Late       |
| <code>get_waiting_list()</code>            | Get files pending download               |
| <code>update_status(downloaded)</code>     | Update status after download             |
| <code>get_status_summary()</code>          | Get counts by status                     |
| <code>cleanup_old_entries(days)</code>     | Remove entries older than N days         |



## 5.4 Cron Integration

MET maintenance runs hourly via cron:

Listing 5.1: Cron Job for MET Maintenance

```
1 # Add to crontab (crontab -e)
2 0 * * * * /path/to/.venv/bin/pygnss-rt maintain-met
3
4 # Or run manually
5 pygnss-rt maintain-met --late-day 3 --late-hour 1
```

## Chapter 6

# Daily PPP Processing

### 6.1 Processing Workflow

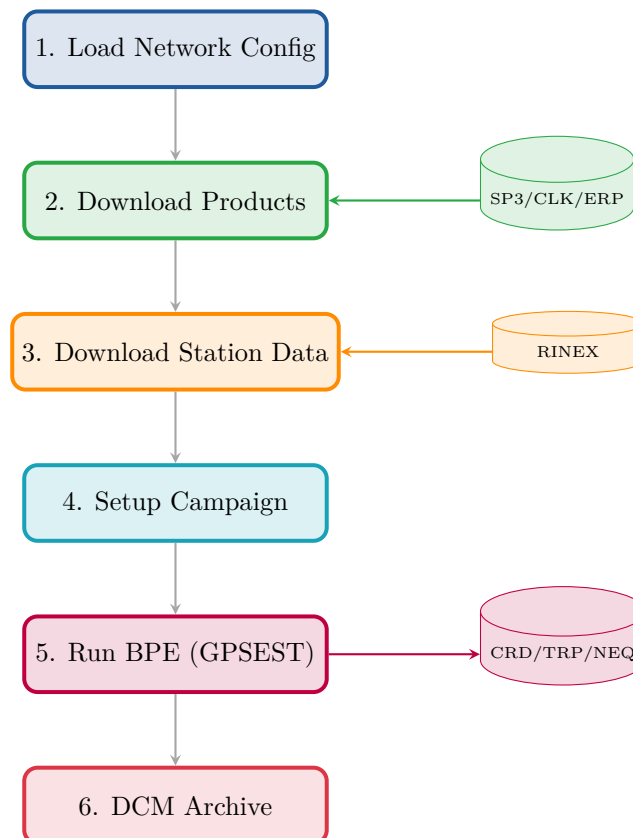


Figure 6.1: Daily PPP Processing Workflow

### 6.2 Running Daily PPP

Listing 6.1: Daily PPP Command Examples

```
1 # Process IGS network for a specific date
2 pygnss-rt daily-ppp IG -s 2025-12-23 -e 2025-12-23
3
4 # Process date range
5 pygnss-rt daily-ppp EU -s 2025-12-01 -e 2025-12-31
6
7 # Process with specific stations
8 pygnss-rt daily-ppp GB -s 2025-12-23 --stations HERS,EUSK,NEWL
9
10 # Exclude stations
11 pygnss-rt daily-ppp RG -s 2025-12-23 --exclude TLSE,MARS
```

```

12
13 # Skip product download (already have them)
14 pygnss-rt daily-ppp IG -s 2025-12-23 --skip-products
15
16 # Dry run (show what would be done)
17 pygnss-rt daily-ppp SS -s 2025-12-23 --dry-run

```

### 6.3 Command Options

Table 6.1: daily-ppp Command Options

| Option                       | Type       | Description                    |
|------------------------------|------------|--------------------------------|
| <code>NETWORK</code>         | Choice     | Network ID: IG, EU, GB, RG, SS |
| <code>-s, -start-date</code> | YYYY-MM-DD | Start date                     |
| <code>-e, -end-date</code>   | YYYY-MM-DD | End date                       |
| <code>-S, -stations</code>   | String     | Comma-separated station list   |
| <code>-x, -exclude</code>    | String     | Stations to exclude            |
| <code>-skip-products</code>  | Flag       | Skip product download          |
| <code>-skip-data</code>      | Flag       | Skip station data download     |
| <code>-skip-dcm</code>       | Flag       | Skip DCM archiving             |
| <code>-dry-run</code>        | Flag       | Show actions without executing |
| <code>-verbose</code>        | Flag       | Verbose output                 |

### 6.4 Result Structure

Table 6.2: DailyPPPResult Fields

| Field                           | Type      | Description                       |
|---------------------------------|-----------|-----------------------------------|
| <code>network_id</code>         | str       | Network identifier (IG, EU, etc.) |
| <code>session_name</code>       | str       | Session name (e.g., 25357IG)      |
| <code>date</code>               | GNSSDate  | Processing date                   |
| <code>success</code>            | bool      | Processing success flag           |
| <code>stations_processed</code> | int       | Number of stations processed      |
| <code>stations_total</code>     | int       | Total stations attempted          |
| <code>duration_seconds</code>   | float     | Processing duration               |
| <code>error_message</code>      | str       | Error details if failed           |
| <code>output_files</code>       | list[str] | Generated output files            |

## Chapter 7

# FTP/Download System

### 7.1 Multi-Source Architecture

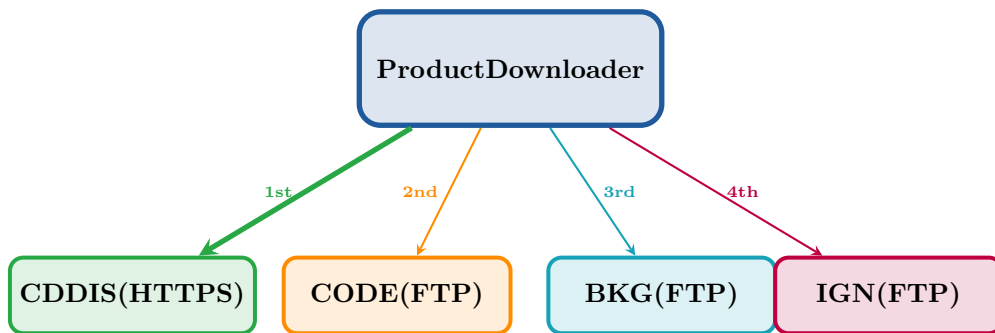


Figure 7.1: Multi-Source Download Architecture

### 7.2 FTP Configuration (YAML)

Server definitions are in `config/ftp_servers.yaml`:

Listing 7.1: FTP Server Configuration

```
1 product_servers:
2   CDDIS:
3     description: "NASA_CDDIS_Primary_IGS_data_center"
4     protocol: https
5     host: cddis.nasa.gov
6     auth_required: true # Uses ~/.netrc
7     products:
8       orbit:
9         final:
10            path: /archive/gnss/products/{gps_week}
11            filename: IGS00PSFIN_{year}{doy:03d}0000_01D_15M_ORB.SP3.gz
12
13   CODE:
14     description: "CODE_Analysis_Center_Bernese_products"
15     protocol: ftp
16     host: ftp.aiub.unibe.ch
17     auth_required: false
18     products:
19       bia:
20         final:
21            path: /CODE/{year}
22            filename: COD00PSFIN_{year}{doy:03d}0000_01D_01D_0SB.BIA.gz
23
24   VMF3:
25     description: "Vienna_Mapping_Functions_TU_Wien"
26     protocol: https
```

```

27  host: vmf.geo.tuwien.ac.at
28  grid:
29      path: /trop_products/GRID/5x5/VMF3/VMF3_OP/{year}
30      filename: VMF3_{year}{month:02d}{day:02d}.H{hour:02d}

```

### 7.3 Supported Products

Table 7.1: GNSS Product Types

| Product | Format | Primary Source | Purpose                       |
|---------|--------|----------------|-------------------------------|
| Orbit   | SP3    | CDDIS          | Satellite positions           |
| Clock   | CLK    | CDDIS          | Satellite clock corrections   |
| ERP     | ERP    | CDDIS          | Earth rotation parameters     |
| BIA/OSB | BIA    | CODE           | Signal biases for PPP-AR      |
| ION     | ION    | CODE           | Higher-order ionosphere       |
| VMF3    | GRD    | TU Wien        | Troposphere mapping functions |

### 7.4 Station Data Sources

Table 7.2: Station Data Servers

| Server    | Protocol | RINEX        | Coverage          |
|-----------|----------|--------------|-------------------|
| CDDIS     | HTTPS    | v3 (.crx.gz) | Global IGS        |
| BKG IGS   | FTP      | v2 (.Z)      | Global IGS mirror |
| BKG EUREF | FTP      | v2 (.Z)      | European stations |
| OSGB      | FTP      | v2 (.gz)     | UK stations       |
| RGP       | FTP      | v2 (.gz)     | French stations   |
| SOPAC     | FTP      | v2 (.Z)      | Global archive    |

## Chapter 8

# Date/Time Handling

### 8.1 GNSSDate Class

The `utils/dates.py` module provides the `GNSSDate` class for unified GNSS time representations:

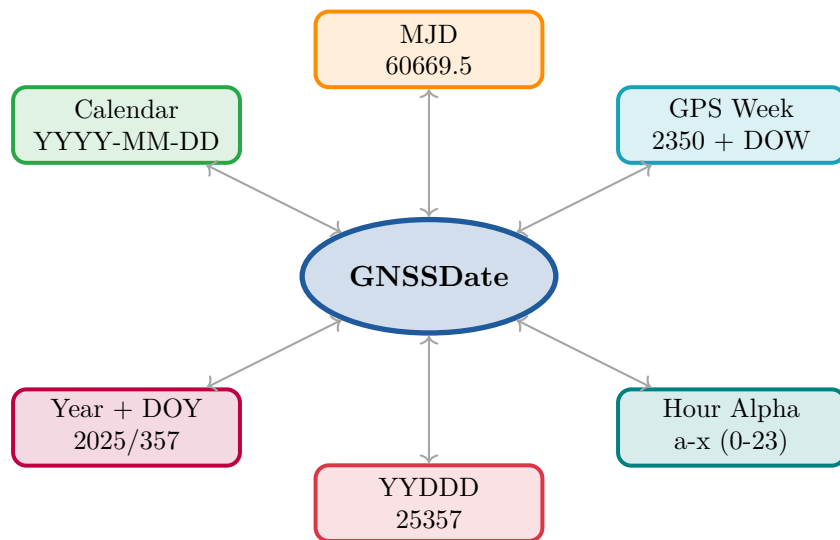


Figure 8.1: GNSSDate Time Representations

### 8.2 Time System Constants

Table 8.1: GNSS Time Constants

| Constant      | Value     | Description                |
|---------------|-----------|----------------------------|
| GPS_EPOCH_MJD | 44244.0   | GPS epoch: January 6, 1980 |
| MJD_OFFSET    | 2400000.5 | Offset from JD to MJD      |



## Chapter 9

# Station Management

### 9.1 Station Class

The `stations/station.py` module provides unified station management:

Table 9.1: Station Class Attributes

| Attribute                  | Type  | Description            |
|----------------------------|-------|------------------------|
| <code>station_id</code>    | str   | 4-character station ID |
| <code>name</code>          | str   | Station name           |
| <code>network</code>       | str   | Network identifier     |
| <code>latitude</code>      | float | Latitude (degrees)     |
| <code>longitude</code>     | float | Longitude (degrees)    |
| <code>height</code>        | float | Ellipsoidal height (m) |
| <code>x, y, z</code>       | float | ECEF coordinates (m)   |
| <code>receiver_type</code> | str   | Receiver type          |
| <code>antenna_type</code>  | str   | Antenna type           |
| <code>use_nrt</code>       | bool  | NRT processing enabled |
| <code>active</code>        | bool  | Station active flag    |

### 9.2 Station File Formats

Table 9.2: Supported Station File Formats

| Format       | Extension   | Notes                              |
|--------------|-------------|------------------------------------|
| YAML         | .yaml, .yml | <b>Preferred</b> – Human-readable  |
| XML          | .xml        | Legacy format – full compatibility |
| BSW STA      | .STA        | Bernese station information file   |
| IGS Site Log | .log        | IGS standard site log format       |

### 9.3 Station Commands

Listing 9.1: Station Management Commands

```
1 # List stations from YAML file
2 pygnss-rt stations /path/to/IGS20.yaml
3
4 # Filter by network
5 pygnss-rt stations stations.yaml --network IGS20
6
7 # Update BSW STA file
8 pygnss-rt update-sta -n IG --output NEWNRT54.STA
9
```



```
10 # Parse IGS site log  
11 pygnss-rt parse-sitelog /path/to/station.log
```

## Chapter 10

# Troposphere Processing

### 10.1 ZTD to IWV Conversion

The `atmosphere/ztd2iwv.py` module converts GNSS-derived ZTD to meteorologically useful IWV:

#### 10.1.1 Mathematical Formulation

The Zenith Total Delay decomposition:

$$\text{ZTD} = \text{ZHD} + \text{ZWD} \quad (10.1)$$

Zenith Hydrostatic Delay (Saastamoinen, 1972):

$$\text{ZHD} = \frac{0.0022768 \cdot P_s}{1 - 0.00266 \cos(2\phi) - 0.00028H} \quad (10.2)$$

Integrated Water Vapor (Bevis, 1994):

$$\text{IWV} = \frac{10^6 \cdot \text{ZWD}}{\rho_w \cdot R_v \cdot (k'_2 + k_3/T_m)} \quad (10.3)$$

Mean atmospheric temperature (Bevis empirical):

$$T_m = 70.2 + 0.72 \cdot T_s \quad (10.4)$$

### 10.2 Physical Constants

Table 10.1: Physical Constants (Bevis, 1994)

|                      | Symbol      | Value    | Unit                | Description                |
|----------------------|-------------|----------|---------------------|----------------------------|
|                      | $R_{dry}$   | 287.0586 | J/kg/K              | Gas constant (dry air)     |
|                      | $R_{vapor}$ | 461.525  | J/kg/K              | Gas constant (water vapor) |
| 2lightpurple!30white | $k_1$       | 77.6     | K/hPa               | Refractivity constant      |
|                      | $k_2$       | 70.4     | K/hPa               | Refractivity constant      |
|                      | $k_3$       | 373900   | K <sup>2</sup> /hPa | Refractivity constant      |
|                      | $g$         | 9.80665  | m/s <sup>2</sup>    | Standard gravity           |
|                      | $\alpha$    | -0.0065  | K/m                 | Temperature lapse rate     |

### 10.3 ZTD2IWV Command

Listing 10.1: ZTD to IWV Conversion

```
1 # Convert single TRP file
2 pygnss-rt ztd2iwv /path/to/troposphere.TRP
3
4 # With MET data for accurate conversion
5 pygnss-rt ztd2iwv tropo.TRP --met-file surface.MET
6
7 # Output to CSV
8 pygnss-rt ztd2iwv tropo.TRP --output iwv_results.csv
```

# Chapter 11

## Bernese Configuration

### 11.1 GRE Mode Configuration

The BSW YAML configuration enables GPS + GLONASS + Galileo processing:

Listing 11.1: GRE Mode Configuration

```
1  # PyGNSS-RT BSW Configuration for PPP-AR (GRE Mode)
2  processing_mode: PPP_AR
3  gnss_systems:
4    GPS: true
5    GLONASS: true
6    Galileo: true
7
8  bern_options:
9    PPP_GEN:
10     CODSPP:
11       # GPS-only for clock sync (BSW 5.4 CODSPP limit)
12       USE_G: '1'
13       USE_R: '0' # Disabled in CODSPP
14       USE_E: '0' # Disabled in CODSPP
15
16     GPSEST:
17       # Full GRE for PPP-AR processing
18       USE_G: '1'
19       USE_R: '1'
20       USE_E: '1'
21       USE_C: '0' # BeiDou not yet supported
22
23     # Troposphere modeling
24     TROPOS: DRY_VMF3
25     TROPOS_S: MENDES-PAVLIS
26
27     # Ambiguity resolution
28     AMBSTRAT: SIGMA
29     USEAR_G: '1'
30     USEAR_E: '1'
31     USEAR_R: '1'
32     GLORES: ALWAYS
33     LAMRES: GNSS_BY_GNSS
34
35     # OSB configuration
36     OSBREC_G: SYSTEM
37     OSBREC_E: SYSTEM
38     OSBREC_R: SATELLITE # Per-satellite for GLONASS FDMA
```

## 11.2 GRE Configuration Rationale

Table 11.1: GRE Processing Strategy

| 2lightgreen!30white | Setting     | Value        | Rationale                     |
|---------------------|-------------|--------------|-------------------------------|
|                     | CODSPP GNSS | GPS-only     | BSW 5.4 CODSPP 2-GNSS limit   |
|                     | GPSEST GNSS | GRE          | Full triple-constellation     |
|                     | GLORES      | ALWAYS       | Enable GLONASS AR across FDMA |
|                     | LAMRES      | GNSS_BY_GNSS | Independent AR per system     |
|                     | OSBREC_R    | SATELLITE    | Per-satellite GLONASS IFBs    |
|                     | TROPOS      | VMF3         | Vienna Mapping Functions      |

## Chapter 12

# Mathematical Foundations

### 12.1 PPP-AR Observation Model

Ionosphere-free linear combination:

$$L_{IF} = \frac{f_1^2 L_1 - f_2^2 L_2}{f_1^2 - f_2^2} \quad (12.1)$$

$$P_{IF} = \frac{f_1^2 P_1 - f_2^2 P_2}{f_1^2 - f_2^2} \quad (12.2)$$

### 12.2 State Vector

$$\mathbf{x} = \begin{bmatrix} \mathbf{r} \\ \delta t_r \\ \text{ZTD} \\ G_N, G_E \\ \mathbf{N} \end{bmatrix} \quad (12.3)$$

### 12.3 LAMBDA Algorithm

Integer ambiguity search minimizes:

$$\|\hat{\mathbf{a}} - \mathbf{a}\|_{\mathbf{Q}_{\hat{\mathbf{a}}}}^2 = (\hat{\mathbf{a}} - \mathbf{a})^T \mathbf{Q}_{\hat{\mathbf{a}}}^{-1} (\hat{\mathbf{a}} - \mathbf{a}) \quad (12.4)$$

Ratio test:

$$R = \frac{\|\hat{\mathbf{a}} - \mathbf{a}_2\|^2}{\|\hat{\mathbf{a}} - \mathbf{a}_1\|^2} > R_{threshold} \quad (12.5)$$

### 12.4 VMF3 Mapping Functions

$$m_h(e) = \frac{1 + \frac{a_h}{1 + \frac{b_h}{1 + c_h}}}{\sin e + \frac{a_h}{\sin e + \frac{b_h}{\sin e + c_h}}} \quad (12.6)$$

# Appendix A

## CLI Command Reference

Table A.1: Complete CLI Commands

| Command                        | Arguments               | Description            |
|--------------------------------|-------------------------|------------------------|
| <code>daily-ppp</code>         | NETWORK -s DATE -e DATE | Run daily PPP-AR       |
| <code>download-products</code> | -d DATE                 | Download GNSS products |
| <code>ztd2iwv</code>           | FILE                    | Convert ZTD to IWV     |
| <code>stations</code>          | FILE                    | List stations          |
| <code>update-sta</code>        | -n NETWORK              | Update BSW STA file    |
| <code>init-db</code>           | -db-path PATH           | Initialize database    |
| <code>maintain-met</code>      | -late-day N             | MET maintenance        |
| <code>db-status</code>         | –                       | Show database status   |

## Appendix B

## References

1. Bevis, M., et al. (1992). GPS Meteorology. *JGR*, 97(D14).
2. Landskron, D., Böhm, J. (2018). VMF3/GPT3. *Journal of Geodesy*, 92.
3. Teunissen, P.J.G. (1995). LAMBDA method. *Journal of Geodesy*, 70.
4. Dach, R., et al. (2015). Bernese GNSS Software v5.2.
5. Schaer, S. (2016). CODE OSB files. *IGS Workshop 2016*.
6. DuckDB Documentation: <https://duckdb.org/docs/>